

ALPS: Admissibility-Preserving Semantic Partitioning for Region-Based Parallel Mining

M Mohaiminul Islam

Department of Computer Science & Telecommunication Engineering

Abstract—Semantic transaction partitioning is not only an optimisation problem, the partition must also work correctly under the execution model. In region-based parallel mining, all regions validate transactions from the same start of round state, so payload clustering can place operations on the same state in different regions. For the independent-barrier model, this paper shows that balance safety for every workload holds if and only if all debits for the same guarded key remain in a single region. Ethereum’s global account nonce creates cross-region ordering dependencies when a sender is split; ALPS enforces sender locality to avoid cross-barrier nonce deferral without intra-round coordination. The proposed static ALPS allocator keeps each sender in one region while optimising payload similarity, active-account interaction cuts, and load under a hard capacity constraint; its chronological variant preserves sender pinning while using load as a soft placement cost. On a 34,326-call Ethereum trace with six regions, transaction k -means splits 1,021 guarded keys and requires 1,068 retry rounds, while ALPS has zero splits and finishes in 27 rounds. Across four disjoint Ethereum/BNB Smart Chain traces with 145,372 calls, ALPS improves retrieval locality and interaction cut over address hashing in all 24 trace/region settings. Compared with balanced sender k -means, ALPS reduces the cut in all 24 settings by 12.8–82.5%; R_{90} improves in 19 settings, ties in two, and trades off in three. Chronological tests preserve sender locality but reveal cold-start costs. The contribution is a scoped safety formalisation and a cost-selectable allocator, not a claim of universal sharding safety or production TPS.

Index Terms—blockchain, parallel mining, transaction partitioning, clustering, state conflicts, Ethereum

I. INTRODUCTION

State-based blockchains usually process transactions in sequence because transaction validity depends on a shared and changing state [1]. Region-based parallel mining [2] reduces this bottleneck by dividing pending transactions among regions. Each region mines independently, and the regional blocks are joined at a barrier into a super-block. The original proposal assigns the initiating account using address modulo the number of regions, but it also allows the region-assignment rule to be configured.

This flexibility creates an opportunity to improve transaction locality. Recent work has explored semantic sharding for blockchain retrieval [3] and workload-aware account placement to reduce cross-shard traffic or adapt to changing workloads [4]–[6]. Conflict-oriented systems instead change cross-shard ordering or scheduling to avoid conflicting execution [7], [8]. These directions motivate smarter placement,

but they do not characterise admissibility for the independent-barrier model considered here. Transactions can be grouped by application payload so that semantically related records stay in the same region, reducing the number of regions scanned by an asset query. However, payload describes an operation, not the state that the operation conditionally consumes. Two miners can therefore validate debits of the same balance against the same stale snapshot. A global account nonce can preserve eventual order through deferral and retry, but splitting a sender creates a long cross-region dependency chain. Therefore, high clustering quality does not guarantee an executable blockchain schedule, while simple address hashing loses useful locality. This paper asks what state-locality invariant is required by the independent-barrier model and how much semantic and interaction locality can be recovered while enforcing it as a hard constraint. The work follows three steps: characterise model-specific admissibility, choose an allocation unit that preserves it, and optimise multiple locality objectives only within that feasible space.

The main contributions of this paper are:

- A model-specific necessary-and-sufficient characterisation of transfer admissibility under independent same-snapshot regional execution, formalising the model’s required single-owner discipline.
- The proposed method, ALPS (Admissibility-preserving Locality and Semantic Partitioning), a sender-indivisible allocator that combines weighted semantic prototypes, an interaction-graph cut, and a load penalty in one normalised objective under a hard static capacity constraint. Exact whole-sender moves provide monotone constrained refinement while safety remains structural; the chronological variant retains sender pinning but treats future load as a soft placement cost.
- A validity-focused evaluation over four non-overlapping direct-call traces (145,372 calls) and a separate 17,445-event log trace. The evaluation includes per-token balance stress, real nonce order, static and chronological tests, component ablations, a frozen-parameter region sweep, and a direct TxAllo allocation comparison. Both improvements and deployment trade-offs are reported.

The scope is limited to the independent-barrier model. Protocols with locks, optimistic validation, cross-shard atomicity,

or re-execution can safely use more general partitions, but they require a coordination mechanism that the model studied here omits.

The rest of this paper is organised as follows. Section II reviews related work. Section III presents the proposed work, including the execution model, admissibility conditions, and the proposed ALPS method. Section IV describes the experimental setup. Section V presents the performance evaluation. Section VI concludes the paper and outlines future work.

II. RELATED WORK

Prior work can be grouped into conflict-aware parallel execution, coordinated sharding, semantic sharding and retrieval, workload-aware placement, and database partitioning. This grouping separates conflict-handling mechanisms from placement objectives and places ALPS within the uncoordinated barrier model.

Chishti and Banerjee [2] introduced the independent regional model studied in this paper. Other approaches alter execution to handle conflicts explicitly. Dickerson *et al.* [9] and Block-STM [10] detect and re-execute conflicts, while static analysis characterises parallelism [11]. ParBlockchain [12] and PEEP [13] encode dependency schedules, Vegeta speculates and replays [14], and Sui Lutris supports object-level independence [15]. Prophet uses Byzantine-tolerant deterministic ordering to avoid conflicting cross-shard execution [7], while the recent Concordia system targets low-conflict distributed transaction scheduling across shards [8]. These mechanisms or programming models are not part of the independent barrier.

Coordinated sharding also changes the execution protocol. SharPer coordinates cross-shard transactions [16], while Monoxide provides eventual atomicity across zones [17]. Therefore, the admissibility condition derived in Section III-A does not restrict these systems; it characterises the cheaper uncoordinated barrier model.

Semantic sharding has separately targeted retrieval and storage locality. SemantiChain allocates transactions using semantic features and semantic distance, and couples that placement with a distributed semantic retrieval architecture [3]. ALPS uses payload semantics for a different purpose: semantic locality is optimised only after the allocation unit has been restricted to preserve admissibility under same-snapshot independent regional execution.

Workload-aware placement is the closest algorithmic line of work. OptChain reduces cross-shard traffic using an online graph [18]; Shard Scheduler places and migrates objects [19]; BrokerChain combines partitioning with broker accounts [20]; and TxAllo jointly optimises traffic and workload balance [4]. More recent work extends account allocation in complementary directions: Orbit uses pending transactions to react to changing traffic before packaging [5], whereas Mosaic moves allocation toward client-driven local optimisation using interaction and workload information [6]. ALPS instead combines retrieval locality, active-account interaction cuts, guarded-state admissibility, and a hard capacity bound in its static allocator. Section V-F directly compares the TxAllo allocation objective.

Since A-TxAllo already places new nodes chronologically, the temporal test in this paper is used as robustness evidence rather than a precedence claim.

Database partitioning provides a broader comparison. Schism balances a co-access graph to reduce distributed transactions [21]. ALPS follows the same general principle but moves only whole senders, making the schedule admissible without abort or retry coordination.

III. PROPOSED WORK

This section first defines the independent-barrier execution model and its admissibility requirements. It then presents the proposed ALPS method for both static and online allocation.

A. System Model and Admissibility

The model imposes a balance-safety locality condition and exposes a separate nonce-ordering cost when senders are fragmented under independent regional execution.

1) *Independent-barrier Execution*: Let T be a transaction set and $P : T \rightarrow \{1, \dots, n\}$ its static region assignment. At the start of round k , every region reads the same committed state σ_k . Each region executes its transactions sequentially on a private copy and accepts a debit only if its local balance is sufficient. Regions do not observe one another during the round. Their debit and credit changes are merged at a barrier to form σ_{k+1} . This follows the execution structure of the base parallel-mining design [2]; proof-of-work is independent of the state argument considered here.

An ERC-20 balance is associated with both an account and a token contract. Let $b = (a, c)$ denote the balance of account a in token contract c . A transfer $t = (a, d, c, x, q)$ conditionally consumes x from debit key $g(t) = (a, c)$, credits (d, c) , and carries account-wide nonce q . Credits are unconditional additions, whereas debits are guarded non-commutative operations.

For balance safety, all transactions that debit the same guarded key must stay in the same region. This requirement, called *guarded-key locality*, can be written as

$$g(t_i) = g(t_j) \implies P(t_i) = P(t_j).$$

A partition is considered barrier-admissible if, for every non-negative starting balance vector and every transfer workload, independently validated regional changes cannot merge into a negative balance.

Theorem 1 (Barrier Admissibility). *Under independent same-snapshot regional execution, a static transfer partition is barrier-admissible for every non-negative starting balance vector and every transfer workload if and only if it satisfies guarded-key locality.*

Proof. For sufficiency, consider a guarded key b in round k . Guarded-key locality implies that only one region r can debit b . Sequential local validation in that region gives

$$D_r(b) \leq \sigma_k(b) + C_r(b),$$

where $D_r(b)$ and $C_r(b)$ are the debit and credit totals of region r . After the regional changes are merged,

$$\sigma_{k+1}(b) = \sigma_k(b) + \sum_j C_j(b) - D_r(b) \geq \sum_{j \neq r} C_j(b) \geq 0.$$

Thus, the merged balance cannot become negative, and the same argument applies inductively over subsequent rounds.

For necessity, suppose two regions debit the same guarded key in one round with positive debit totals d_1 and d_2 , and no credits are made to that key. If the starting balance is $\max(d_1, d_2)$, both regions can accept their debits because they validate against the same starting state. After the regional changes are merged, the balance becomes

$$\max(d_1, d_2) - d_1 - d_2 < 0.$$

Hence any partition that violates guarded-key locality fails to be barrier-admissible for some valid starting balance and workload. $\square$

This result is specific to the independent-barrier model. For example, Monoxide provides asynchronous cross-zone transaction semantics [17], while Block-STM detects conflicts and re-executes transactions [10]. These systems therefore do not follow the uncoordinated merge assumed in this model.

2) *Sender Locality for Global Nonces*: Ethereum's transaction nonce is keyed by account rather than by token. Under the independent-barrier model, splitting one sender across regions creates cross-region ordering dependencies: future-nonce transactions may need to wait for lower nonces committed elsewhere. ALPS therefore enforces sender locality, placing all transactions from sender a , across all tokens, in one region so that nonce order can progress without cross-barrier deferral. Sender locality is stronger than guarded-key locality and preserves both token-balance locality and account order structurally.

If a sender is split across regions, a correct mempool must not permanently reject a future-nonce transaction. Instead, the transaction should be deferred and retried after lower nonces commit. This preserves eventual correctness, but different fragments of the same sender sequence can progress only across barriers, which increases the scheduling cost. Section V evaluates this effect. A separate balance-only execution is also used as a deliberate stress test of the guarded-key locality condition; it is not presented as a replacement for Ethereum's nonce rule.

B. Proposed Method: ALPS

ALPS uses sender-indivisible allocation and combines semantic locality, interaction locality, and load control. The proposed method first performs static allocation and then extends it with online sender pinning.

1) *Allocation Unit and Objective*: ALPS treats each active sender a and its complete transaction sequence as one indivisible unit. Let w_a denote its historical transaction count, x_a its mean payload feature vector, and e_{ab} the number of observed transfers between active senders a and b . Define $W = \sum_a w_a$,

$M = \sum_{\{a,b\}} e_{ab}$, the mean regional load $\bar{L} = W/n$, and $L_r = \sum_{a:z_a=r} w_a$. The exact dimensionless objective is

$$\begin{aligned} J = & \frac{1}{W} \sum_a w_a \frac{\|x_a - \mu_{z_a}\|_2^2}{s} \\ & + \frac{\lambda_g}{M} \sum_{\{a,b\}} e_{ab} \mathbf{1}[z_a \neq z_b] \\ & + \frac{\lambda_l}{n} \sum_{r=1}^n \left(\frac{L_r}{\bar{L}} \right)^2 \end{aligned} \quad (1)$$

The objective is subject to one region per sender and $L_r \leq C_{\text{eff}}$. The target cap is $C_0 = \alpha \bar{L}$. When a whole-sender packing satisfies C_0 , ALPS sets $C_{\text{eff}} = C_0$; otherwise, the LPT fallback supplies a feasible certificate and C_{eff} is relaxed only to the maximum load of that certificate. No reported run required this relaxation, so $C_{\text{eff}} = C_0$ in all reported static results. Here, s is the median nonzero semantic squared distance. The experiments use $\alpha = 1.15$, $\lambda_g = 0.15$, and $\lambda_l = 0.20$. Section V reports the graph-weight frontier rather than treating 0.15 as universally optimal.

The preferred operating point depends on the deployment setting. Let X be the active-account interaction cut defined in Section IV. For a deployment score $D_\kappa = R_{90} + \kappa X$, where κ is the relative penalty assigned to interaction-cut fraction in R_{90} -equivalent units, ALPS is preferable to a baseline B when

$$\kappa > \kappa^* = \frac{R_{90}^{\text{ALPS}} - R_{90}^B}{X_B - X_{\text{ALPS}}} \quad (2)$$

provided $X_{\text{ALPS}} < X_B$. A non-positive threshold means that ALPS weakly dominates. This formulation makes the retrieval–interaction trade-off explicit instead of hiding it inside a single composite score.

The three terms have different roles. The payload-distance term preserves asset-query locality, the graph term reduces active-account cross-region interactions, and the capacity constraint prevents a popular asset or sender community from overloading one miner. Sender indivisibility is enforced outside the objective, so optimisation cannot trade correctness for a lower objective value.

Algorithm 1 ALPS static allocation. Every operation moves a whole sender, so sender locality is an invariant rather than a penalty.

Input: history T_h , regions n , cap multiplier α

Output: one pinned region z_a per active sender

```

1: compute  $(w_a, x_a)$  for active senders and graph weights  $e_{ab}$ 
2:  $C_0 \leftarrow \alpha \bar{L}$  {target per-region capacity}
3: initialise  $\mu_1, \dots, \mu_n$  by transaction-weighted  $k$ -means
4: for senders  $a$  in non-increasing  $w_a$  do
5:    $F \leftarrow \{r : L_r + w_a \leq C_0\}$ , or  $\{\arg \min_r L_r\}$  if empty
6:    $z_a \leftarrow \arg \min_{r \in F} \Delta J(a \rightarrow r)$ ;  $L_{z_a} += w_a$ 
7: end for
8: if  $\max_r L_r > C_0$  then
9:   reseed by LPT packing and recompute  $L$  {feasible certificate}
10: end if
11:  $C_{\text{eff}} \leftarrow \max(C_0, \max_r L_r)$ 
12: repeat
13:   recompute weighted prototypes  $\mu_r$ 
14:   for senders  $a$  in non-increasing  $w_a$  do
15:      $r^* \leftarrow \arg \min_r \Delta J(a : z_a \rightarrow r)$  s.t.  $L_r + w_a \leq C_{\text{eff}}$ 
16:     move all of  $a$  to  $r^*$  and update  $L$  if  $\Delta J < 0$ 
17:   end for
18: until no move accepted or six sweeps
19: return  $\{z_a\}$ 

```

After transaction-weighted k -means initialisation, the implementation assigns heavy senders first and then performs deterministic coordinate refinement with prototype updates (Algorithm 1). If the cost-aware seed violates the target cap C_0 , largest-processing-time-first (LPT) packing provides a feasible whole-sender certificate. The refinement stage then enforces the resulting C_{eff} as a hard constraint. No reported run required relaxation beyond C_0 . The implementation and fixed parameters are included with the artifact.

a) Static Time Complexity: Let $N = |T_h|$, let A denote the number of active senders, let E denote the number of active-sender interaction edges, let I denote the number of weighted k -means iterations, and let $S \leq 6$ denote the number of refinement sweeps. Building sender profiles and the interaction graph requires a pass over the history, costing $O(Nd)$ in general (or $O(N)$ for the fixed, low-dimensional feature used in our experiments). Weighted k -means initialisation costs $O(InAd)$, and sorting senders by weight costs $O(A \log A)$. The capacity-aware seed and each subsequent refinement sweep evaluate candidate regions while accounting for semantic distances and incident interaction edges, giving $O(n(Ad+E))$ per pass. Hence the end-to-end static allocation has worst-case time

$$O(Nd + InAd + A \log A + (S+1)n(Ad+E)). \quad (3)$$

Because S is capped at six, refinement is bounded by a small constant number of passes. If the LPT fallback is invoked, its sorting/packing overhead is $O(A \log A + A \log n)$; no reported run required this fallback. Excluding the input trace itself, the principal allocator storage is $O(Ad+E+n)$ for sender profiles, graph edges, assignments, loads, and prototypes.

Proposition 1 (Monotone Constrained Refinement). *With fixed prototypes, every accepted ALPS move strictly decreases (1)*

while preserving sender locality and the effective capacity constraint. Replacing each prototype by its transaction-weighted mean cannot increase the semantic term.

Proof. For each candidate move, ALPS evaluates the exact change in the sender’s semantic term, its incident interaction-cut edges, and the load terms of the two affected regions. A move is accepted only when the total change is negative and the destination region has sufficient remaining capacity. Because the whole sender moves as one unit, sender locality is preserved. For fixed assignments, the transaction-weighted mean minimises the within-region squared semantic distance. Therefore, accepted moves strictly decrease the objective, while prototype updates cannot increase its semantic term. Since the assignment space is finite, unrestricted refinement reaches a one-move local optimum. The implementation stops after at most six sweeps, so it may stop earlier while still preserving monotonicity and feasibility. $\square$

2) *Online Placement:* A deployable allocator cannot build a sender profile from transactions that have not yet occurred. The temporal version therefore learns prototypes and assignments only from a chronological prefix. A known sender keeps its assigned region. A sender absent from the history is assigned exactly once, when its first future transaction arrives. The assignment uses the transaction’s visible payload, the receiver’s region when the receiver is already placed, and the current history-derived placement-load estimate. The selected region is then pinned.

Algorithm 2 ALPS online placement. A sender is assigned once, at its first appearance, so later actions do not affect the earlier assignment.

Input: history state $\{z_a\}, \{\mu_r\}, \{L_r\}, C_0$; future stream T_f

Output: a pinned region for every sender in T_f

```

1: for transactions  $t \in T_f$  in chronological order do
2:    $a \leftarrow \text{sender}(t)$ 
3:   if  $a$  already has a region then
4:     route  $t$  to  $z_a$  {pinned; never revised}
5:   else
6:      $\rho \leftarrow \text{region of receiver}(t)$  if placed, else  $\perp$ 
7:      $z_a \leftarrow \arg \min_r [\|\text{feat}(t) - \mu_r\|^2/s + \lambda_g \mathbf{1}[r \neq \rho] + \lambda_l (L_r/C_0)^2]$ 
8:     pin  $z_a$ ;  $L_{z_a} \leftarrow L_{z_a} + 1$ ; route  $t$  to  $z_a$ 
9:   end if
10: end for

```

Algorithm 2 uses only information available at the time of placement. Later transactions do not affect earlier sender assignments. Here, L_r is the placement-load estimate carried forward from the historical allocation; an unseen sender contributes one unit when it is first pinned because only its first future transaction is then observable. Transactions from already pinned senders do not revise this placement estimate or their assignments. Unlike the static allocator, the online rule therefore does not enforce $L_r \leq C_0$ as a hard future-stream feasibility constraint: the historical target C_0 acts only as the scale in the load penalty. Sender locality remains structural,

while realised future load can exceed the historical 1.15 target when sender activity changes.

a) *Online Time Complexity*: With a hash-map assignment table, routing a transaction from an already pinned sender takes $O(1)$ expected time. A previously unseen sender is placed only once; evaluating its semantic and load-aware cost over all n regions costs $O(nd)$. Therefore, for a future stream of F transactions containing U previously unseen senders, online placement costs

$$O(F + Und), \quad (4)$$

with the more expensive $O(nd)$ decision paid only at each sender’s first appearance. The assignment map grows by $O(U)$ entries over the future stream.

This temporal split is used as a robustness check rather than a methodological novelty claim. A-TxAllo also uses a chronological 9:1 partition, carries the previous allocation forward, and explicitly places new nodes using newly committed transactions [4]. The evaluation in this paper instead uses four fixed history horizons, pins each sender at its first appearance, and evaluates retrieval together with interaction cut and load.

IV. EXPERIMENTAL SETUP

This section describes the traces and feature construction, the compared schemes and execution settings, and the evaluation metrics and chronological protocol.

A. Traces and Feature Construction

Table I summarises three Ethereum mainnet traces covering eight ERC-20 contracts and one held-out BNB Smart Chain trace with a disjoint set of eight tokens. D1–D3 include only direct `transfer(address,uint256)` calls. Therefore, the outer signer is both the debited token owner and the actual account-nonce owner. Nonces are rank-normalised for each sender without changing their order. The traces cover non-overlapping spans on 14, 13, and 7 August 2026, with D3 separated from the others by one week. D1 contains documented collection gaps, while D2, D3, and C1 scan their full stated spans.

The selector, calldata layout, and nonce semantics are common to both EVM chains. C1 is intentionally challenging for semantic placement because one token accounts for 94.9% of transfers, compared with 69.7–72.7% on Ethereum. A secondary `Transfer-log` trace extends the balance-locality population but cannot identify outer signers for router-mediated calls. It is therefore excluded from nonce-related claims. Exact bounds, filters, hashes, and collection provenance are provided in the artifact manifests.

Collection uses public JSON-RPC endpoints. Complete-scan collectors retain transaction hashes and stop if a block is missing. Eleven self-transfers are excluded from the log trace, leaving 17,445 evaluated events in Table I.

Raw ERC-20 values are not directly comparable across token decimals and price scales. For each token, the raw value is divided by its trace median, clipped to $[0.1, 20]$, and used in the balance stress model. Clustering features combine a scaled

TABLE I
MEASURED TRACES. BLOCKS/SPAN SHOWS REPRESENTED BLOCKS OVER THE INCLUSIVE HEIGHT SPAN. D1 HAS DOCUMENTED GAPS; D2, D3, AND C1 COVER THEIR FULL SPANS. TOP TOKEN IS THE SHARE OF THE MOST FREQUENT CONTRACT.

Trace	Chain	Transfers	Blocks/span	Senders	Top token	Multi-tok.
Direct calls (D1)	ETH	34,326	513/714	18,747	69.7%	19.5%
Prior-day (D2)	ETH	39,425	713/714	20,719	70.0%	20.9%
Prior-week (D3)	ETH	45,893	499/500	27,963	72.7%	19.9%
Cross-chain (C1)	BSC	25,728	2489/2500	14,082	94.9%	10.3%
Transfer logs	ETH	17,445	81/137	7,195	—	—

TABLE II
DESIGN PROPERTIES OF THE COMPARED METHODS. LOCAL DENOTES SENDER/GUARDED-STATE LOCALITY; CAP DENOTES A HARD STATIC CAPACITY CONSTRAINT; COORD DENOTES COORDINATION BEYOND THE INDEPENDENT BARRIER.

Method	Local	Sem.	Cut	Cap	Coord.
ADDR-MOD	Yes	No	No	No	No
TX-KMEANS	No	Yes	No	No	Retry
SENDER-KMEANS	Yes	Yes	No	No	No
S-KM-BAL	Yes	Yes	No	Yes	No
TXALLO	Yes	No	Yes	No	Protocol
ALPS	Yes	Yes	Yes	Yes	No

exact token one-hot vector with $\log(1 + \text{relativeAmount})$. No Gaussian jitter or synthetic clustering coordinate is used. Therefore, when transaction k -means splits a dominant token, the split is based on an observed amount difference.

B. Schemes and Execution

The static evaluation compares all methods except TX-ALLO. ADDR-MOD uses $a \bmod n$, TX-KMEANS clusters individual transactions, and the sender-based variants cluster mean features of active senders. S-KM-BAL additionally uses the $1.15\times$ load target, while ALPS uses the joint objective described in Section III-B. Section V-F separately compares TXALLO.

The evaluation varies $n \in \{2, 3, 4, 6, 8, 11\}$ with a regional block capacity of 250. For the global nonce experiment, each (account, token) debit key is funded with its complete observed outgoing amount plus a cushion. Therefore, both sequential execution and any correct retrying schedule can commit the full trace. Extra rounds reflect ordering and partitioning effects rather than arbitrary balance failures. Each regional mempool selects currently ready sender/nonce entries, commits up to capacity, retains future-nonce transactions, and retries them after the barrier.

The complete static sweep uses deterministic seed 0 with k -means `n_init`=10. The temporal, frontier/ablation, and Tx-Allo comparisons use three initialisations. Allocation-sensitive conclusions are therefore reported from these sensitivity runs as well as the exact static audit.

For the balance-safety experiment, each observed (account, token) key instead receives the same stress balance, varied over $\{1000, 600, 400, 250, 150, 100, 60\}$. Account nonces are ignored only in this experiment to expose simultaneous guarded-key debits. A negative balance after merging is an

exact witness for that run, and token balances are never pooled across contracts.

State and scheduling experiments set proof-of-work difficulty to zero. The evaluation reports barrier rounds and idealised scheduling throughput, $Q_s = N_c/R$, where N_c is the number of committed transactions and R is the number of rounds. This is not machine-dependent wall-clock TPS. With regional capacity B , $Q_s/(nB)$ represents schedule utilisation and isolates dependency stalls from hashing, storage, networking, and consensus. The simulator retains a genuine SHA-256 path for separate benchmarking, but no wall-clock claim depends on it.

C. Metrics and Temporal Protocol

The evaluation reports split senders and guarded keys, transactions issued by split senders, barrier rounds, deferred nonce checks, maximum-to-mean regional load, and the active-account interaction cut. For a given allocation, let T_{act} be transfers whose receiver also appears as an active sender. The interaction cut is

$$X = \frac{|\{t \in T_{\text{act}} : z_{\text{sender}(t)} \neq z_{\text{receiver}(t)}\}|}{|T_{\text{act}}|},$$

and interaction coverage is $|T_{\text{act}}|/|T|$. Receiver-only addresses are therefore excluded from the cut rather than automatically counted as cross-region. Regional load is reported as $\max_r L_r/\bar{L}$. For semantic retrieval locality, regions are ranked for each token by that token's transfer count; R_{90} is the number of highest-ranked regions required to cover at least 90% of that token's transfers, averaged over the evaluated tokens. Thus, lower R_{90} means fewer regions need to be scanned, not lower measured query latency.

Static interaction coverage depends only on the trace and is therefore strategy-invariant: 53.8% for D1, 56.6% for D2, 60.6% for D3, and 45.7% for C1. This conditional metric matches ALPS's active-sender graph while keeping the denominator explicit.

The online evaluation uses $n = 6$, chronological history fractions $\{0.5, 0.6, 0.7, 0.8\}$, and three k -means initialisations, giving 12 history/future configurations per method. Figures report the mean and standard deviation across these configurations as sensitivity summaries rather than as independent repeated chain samples. Across the four traces, 60.0% of future transactions come from senders absent from the training prefix, making cold-start handling important.

V. PERFORMANCE EVALUATION

The evaluation examines safety, retry cost, locality, temporal robustness, component effects, and comparison with TxAllo. The headline parameters use $\lambda_g = 0.15$ and $\alpha = 1.15$, selected using D1 as the development trace and then held fixed; D2, D3, and C1 provide out-of-sample robustness checks. The static and head-to-head evaluations cover all six region counts, while the stress, temporal, and mechanism analyses use $n = 6$. The operating frontier varies only λ_g , and each ablation removes one component at a time. Overall, the static sweep contains

120 trace/region/scheme measurements, and the balance-stress experiment contains 175 measurements (140 sender-local and 35 TX-KMEANS).

A. Admissibility under Payload Partitioning

Payload-based partitioning violates sender locality on every trace and, at larger region counts, also violates guarded-key locality. Across the 96 sender-local measurements—ADDR-MOD, both sender k -means variants, and ALPS over four traces and six region counts—the number of split guarded keys, split senders, and overdrawn accounts is exactly zero, and every execution commits the complete trace. In contrast, TX-KMEANS splits 66–1,592 senders and up to 1,334 guarded keys. Transactions issued by split senders account for 10.3–44.4% of each trace (Fig. 1(a)). Table III reports the detailed results for $n = 6$.

The distinction between guarded-key locality and sender locality is clear at $n = 2$. Across all four traces, TX-KMEANS splits no guarded keys but still splits 66–551 senders. With two regions, clustering places complete token populations on each side, so every (account, token) balance remains region-local and balance safety is preserved. However, accounts that transact across multiple tokens can still be divided between regions, creating cross-region dependencies for the account-wide nonce. This result directly shows the difference between guarded-key locality and sender locality and is visible because balances are modelled separately for each token.

The balance-stress experiment further confirms this behaviour. Across 175 measurements—five traces, seven starting balances, and five schemes—TX-KMEANS overdraws 6–48 accounts in each of its 35 rows, while all 140 sender-local rows record zero overdraws. The lowest-balance tail is non-monotone because tighter balances can cause regional transfers to fail before two conflicting debits are both accepted and merged. Therefore, the number of witnesses is not expected to increase monotonically.

B. Retry Cost of Sender Fragmentation

Deferring and retrying future-nonce transactions restores eventual commit, but it does not remove the scheduling cost. Every scheme commits 100% of every trace with no pending transactions once future nonces are retried. However, TX-KMEANS requires 156–1,760 rounds, compared with 11–106 for ALPS, and performs up to 3.13 million future-nonce checks. Its scheduling throughput, Q_s , falls to 22–253 committed transactions per round, compared with 432–2,338 for ALPS.

At $n = 6$, ALPS achieves $40\times$, $45\times$, $23\times$, and $24\times$ higher scheduling throughput than TX-KMEANS on D1, D2, D3, and C1, respectively (Fig. 1(b), Table III). The reason is that preserving a global nonce order for fragmented senders forces progress to continue across barriers, creating a strong scheduling penalty on every trace.

C. Locality and Load Trade-offs

Relative to ADDR-MOD, enforcing sender locality does not prevent ALPS from improving both locality metrics on

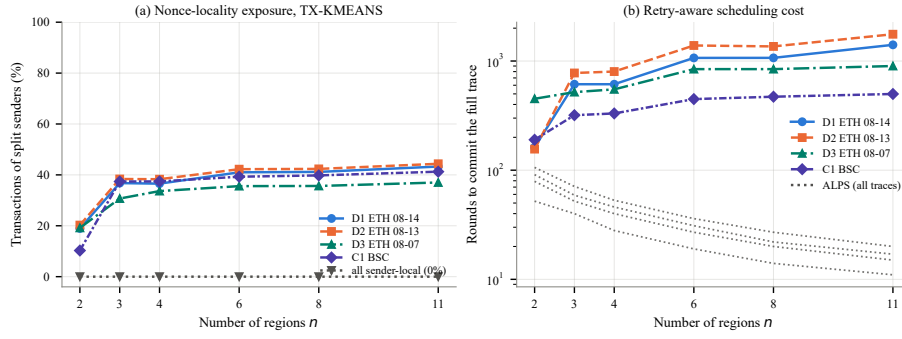


Fig. 1. Locality violations and retry-aware scheduling cost across the four direct-call traces. Sender-local schemes have zero split-sender exposure in (a), while (b) compares TX-KMEANS and ALPS round counts.

TABLE III

RESULTS AT $n = 6$ FOR ALL DIRECT-CALL TRACES. SPLIT KEYS ARE GUARDED (ACCOUNT, TOKEN) KEYS; OVERDRAWS SHOW THE RANGE ACROSS THE SEVEN-LEVEL BALANCE STRESS. LOWER IS BETTER EXCEPT Q_s .

Method	Split keys	Rounds	Q_s	Over-drawn	R_{90}	Load	Active cut
D1 ETH 08-14							
ADDR-MOD	0	33	1,040	0	5.88	1.43	.837
TX-KMEANS	1,021	1,068	32	19–48	1.38	2.27	.425
S-KM-BAL	0	27	1,271	0	2.25	1.15	.646
ALPS	0	27	1,271	0	3.00	1.15	.260
D2 ETH 08-13							
ADDR-MOD	0	37	1,066	0	5.88	1.39	.841
TX-KMEANS	1,061	1,388	28	20–48	1.38	2.28	.432
S-KM-BAL	0	31	1,272	0	2.38	1.15	.670
ALPS	0	31	1,272	0	2.88	1.15	.279
D3 ETH 08-07							
ADDR-MOD	0	38	1,208	0	5.50	1.22	.834
TX-KMEANS	1,067	843	54	12–36	1.38	2.08	.614
S-KM-BAL	0	36	1,275	0	3.12	1.15	.818
ALPS	0	36	1,275	0	2.88	1.15	.528
C1 BSC							
ADDR-MOD	0	21	1,225	0	4.62	1.21	.837
TX-KMEANS	478	448	57	13–34	1.25	2.93	.413
S-KM-BAL	0	20	1,286	0	2.62	1.15	.859
ALPS	0	19	1,354	0	2.00	1.07	.236

these traces; the additional cost is allocator computation (Section V-E). ALPS improves both locality metrics over ADDR-MOD in all 24 trace/region settings. R_{90} decreases by 18.8–68.5% (mean 45.9%), while the interaction cut decreases by 35.2–75.9% (mean 57.7%), with static load never exceeding the 1.15 target.

A more informative comparison is with balanced sender k -means, which is already admissible. ALPS reduces the interaction cut in all 24 settings by 12.8–82.5% (mean 50.9%). It also improves R_{90} in 19 settings, ties in two, and regresses in three. These three retrieval-locality trade-offs occur at $n = 6$ on D1 and D2 and at $n = 4$ on D3; only the D1 case is on the development trace used for parameter selection. We therefore report these exceptions explicitly rather than claiming universal dominance. In most settings, ALPS provides a strict improvement; otherwise, the break-even ratio κ^* identifies when the simpler scheme is preferable under the score in (2).

For the D1 $n = 6$ setting, $\kappa^* = 1.94$: weights above this value favour ALPS because they place greater relative penalty on interaction cut, whereas smaller weights favour SENDER-KMEANS-BAL. At higher region counts, ALPS often weakly dominates, making the threshold non-positive.

Figure 2 shows why these exceptions are limited to small n . The retrieval cost of ADDR-MOD increases sharply with the number of regions, giving semantic partitioning more room for improvement. At $n = 11$, ALPS improves R_{90} over balanced sender k -means by 10.0–37.8% across all four traces. Unconstrained SENDER-KMEANS achieves strong semantic locality but becomes highly imbalanced, reaching a load of 4.45 at $n = 11$. In contrast, the capacity-aware schemes never exceed 1.15.

D. Temporal Robustness

Chronological evaluation preserves safety but weakens locality. Every online assignment records zero split senders because each unseen sender is assigned once and then pinned.

Compared with balanced sender k -means, ALPS reduces the future interaction cut by 19.3%, 23.7%, 37.9%, and 14.1% on D1, D2, D3, and C1, respectively, while also lowering the observed load (Fig. 3). Retrieval shows a mixed result: R_{90} is 4.8% and 8.0% higher on D1 and D2, nearly unchanged on D3, and 30.3% lower on C1. Unconstrained SENDER-KMEANS achieves lower interaction cuts on D3 and C1 (.327 and .576, compared with .513 and .745 for ALPS), but with much higher loads of 3.68 and 2.58, compared with 1.48 and 1.43 for ALPS.

C1 shows the weakest interaction-cut improvement. ALPS improves over hashing by 11.9% on C1, compared with 28.8–38.7% on the Ethereum traces. One token accounts for 94.9% of C1 transfers, and 62.2% of future traffic comes from unseen senders. As a result, first-transaction payloads provide limited information for distinguishing accounts. Sender pinning still preserves safety, but the online allocator uses the static 1.15 target only as a load-cost scale and does not impose it as a hard future-load constraint; pinned-sender activity can therefore drive observed future load above 1.15.

The causal interaction cut includes only receiver placements available at the time of arrival, giving coverage of 26.4–

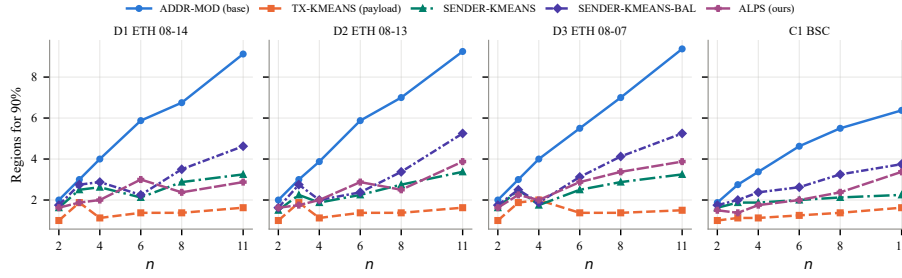


Fig. 2. Semantic retrieval cost across all direct-call traces. TX-KMEANS is an optimistic lower bound because it violates the execution invariant; the other methods are sender-local.

44.3% across the traces. Unplaced receivers are reported separately. Overall, 60.0% of future traffic comes from unseen senders, which explains much of the gap between static and chronological performance.

E. Component Analysis and Allocator Overhead

The same component effects appear across all four traces. Removing the semantic term degrades retrieval locality, with R_{90} increasing to 4.12–5.25, at or above the address-hashing level, although the interaction cut decreases as expected. Removing the capacity cap increases load to 1.28–1.52, exceeding the measured 1.15 maximum-imbalance target. Setting $\lambda_g = 0$ removes the interaction term and produces a semantic-plus-load ALPS variant. This variant is close to, but not identical to, balanced sender k -means because ALPS uses transaction-weighted prototypes and exact objective refinement. Comparing it with positive graph weights isolates the effect of the interaction term without assuming algebraic equivalence to that baseline.

Figure 4 shows the operating curve for each trace. The fixed $\lambda_g = 0.15$ lies in the low-cut region of every measured curve, although no universal knee is claimed. C1 begins to flatten earlier than the three Ethereum traces. Beyond 0.35, the curve may flatten or reverse, which is consistent with the non-convex discrete search.

Allocator overhead remains modest at the measured scale. On D1 prefixes of 5,000, 10,000, 20,000, and 34,326 calls, fresh single-worker runs over three seeds require median profile-plus-allocation times of 0.89, 1.83, 5.22, and 9.94 s, respectively, on an Intel i9-14900. At the full trace, with 18,747 active senders and 16,120 interaction edges, the peak process working set is 226.2 MiB, and no run relaxes the capacity bound. These measurements represent allocator overhead, not production TPS.

F. Comparison with TxAllo

The closest algorithmic comparison is TxAllo [4], which allocates accounts using community detection and jointly optimises cross-shard traffic and workload balance. This paper reimplements its allocation objective using Louvain initialisation, candidate-community moves, and coordinate descent on its throughput score. Protocol components, including consensus, settlement, brokers, and the A-TxAllo epoch update,

TABLE IV
ALPS AND THE REIMPLEMENTED TXALLO ALLOCATION OBJECTIVE. VALUES ARE THREE-SEED MEANS WITH FIXED WEIGHTS; BOTH METHODS RECORD ZERO SPLITS.

Trace	n	TXALLO (reimpl.)			ALPS		
		R_{90}	Cut	Load	R_{90}	Cut	Load
D1	4	3.50	.108	1.01	2.00	.394	1.15
D1	11	7.33	.166	1.16	2.88	.314	1.14
D2	4	3.62	.116	1.01	2.00	.390	1.15
D2	11	7.50	.186	1.21	3.75	.320	1.15
D3	4	3.67	.101	1.03	2.00	.487	1.15
D3	11	7.75	.211	1.67	3.83	.444	1.15
C1	4	3.08	.177	1.01	1.83	.205	1.07
C1	11	5.71	.253	1.18	3.25	.349	1.15

are excluded because they change the execution semantics. Therefore, Table IV compares the allocation objectives under the same execution model rather than deployed throughput. The comparison uses $\eta = 2$ and eight coordinate sweeps.

TxAllo is account-keyed and records zero split senders and guarded keys in all 24 settings. Thus, this independently designed workload-aware allocator also satisfies the locality requirement described in Section III-A. The two methods favour different objectives. Relative to ALPS, TxAllo reduces the active-account cut in all 24 settings by 10.2–82.5% (mean 52.0%). In contrast, ALPS reduces R_{90} in all 24 settings by 16.3–60.8% (mean 39.6%). Thus, TxAllo is better suited to deployments that assign a high relative penalty to active-account cross-region interaction, while ALPS favours retrieval-locality-sensitive deployments. At $n = 11$, TxAllo also permits loads of 1.16–1.67, whereas static ALPS maintains its hard bound at or below 1.15.

A one-factor sensitivity analysis at $n = 6$ varies $\eta \in \{1.25, 1.5, 2, 3, 4\}$ with eight sweeps and varies the sweep cap over $\{4, 8, 16, 32\}$ with $\eta = 2$, across four traces and three seeds. ALPS retains lower R_{90} in all 12 trace/seed settings for every value of η . TXALLO retains the lower interaction cut in all 12 settings through $\eta = 3$ and in 10 of 12 settings at $\eta = 4$. It exceeds the 1.15 load target once at each of $\eta = 3$ and $\eta = 4$. All four sweep caps produce identical metrics, indicating convergence within four sweeps in these settings.

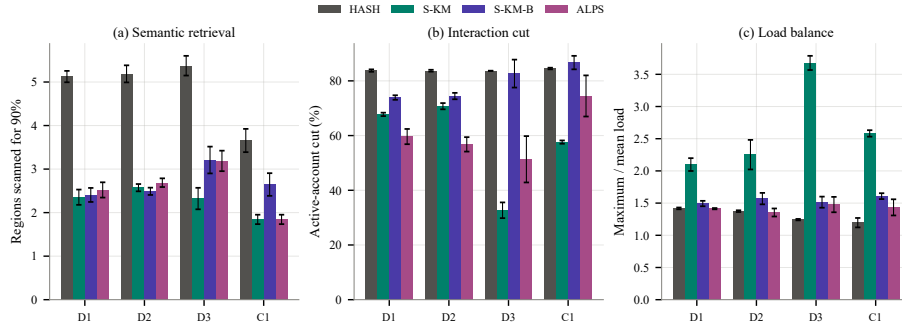


Fig. 3. Chronological evaluation at $n = 6$. Bars show means over four history fractions and three initialisations; error bars show standard deviations. S-KM and S-KM-B denote sender k -means and its balanced variant.

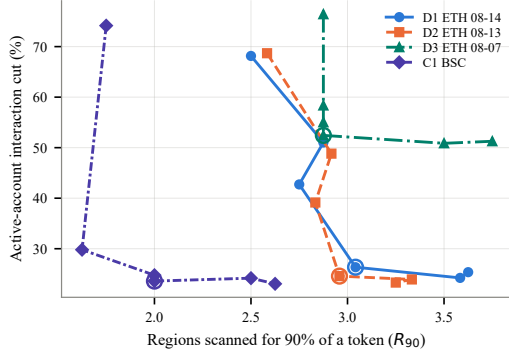


Fig. 4. ALPS retrieval–interaction trade-off across all direct-call traces. The fixed $\lambda_g = 0.15$ is circled; lower values are better on both axes.

VI. CONCLUSION AND FUTURE WORK

Semantic partitioning is not only an optimisation problem; the resulting allocation must also be executable. Under independent parallel mining, guarded-key locality is required for balance safety, while splitting a sender creates cross-region nonce-ordering dependencies; ALPS enforces sender locality to eliminate the resulting cross-barrier deferral. Retry-aware transaction clustering can eventually commit future-nonce transactions, but the 1,068-round schedule at six regions shows that eventual correctness alone does not guarantee efficient execution. The proposed ALPS method combines this model-specific admissibility requirement with sender-indivisible allocation and then optimises retrieval locality, active-account interaction cut, and load within the feasible space. Static ALPS enforces a hard effective capacity bound, equal to the 1.15 target in every reported run, whereas the chronological variant preserves sender locality while using the historical target only as a soft load-placement scale, so future activity drift can exceed it. The evaluation shows a clear trade-off between retrieval locality and active-account cross-region interaction, with an explicit deployment boundary. The evaluation is limited to relatively short traces from two EVM chains, with three Ethereum traces sharing the same contract set; direct-call traces also exclude router-mediated transactions, historical balances are not reconstructed, and

the TxAllo comparison covers its allocation objective rather than the complete protocol. In addition, R_{90} measures region-scan locality rather than query latency, and the reported round-based throughput measures scheduling efficiency rather than production TPS. Future work will consider longer and more diverse multi-chain traces, archive-state replay, general contract read/write sets, epoch reallocation and migration, and full-protocol integration.

REFERENCES

- [1] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” Ethereum Yellow Paper, 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [2] M. S. Chishti and A. Banerjee, “Increasing TPS rate of state-based blockchains by parallel mining,” *Internet Technology Letters*, vol. 4, no. 2, p. e220, 2021.
- [3] Z. Zhen, X. Wang, X. Yang, J. Shu, J. Hu, H. Lin, and X. Yi, “SemantiChain: A trust retrieval blockchain based on semantic sharding,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 10 339–10 354, 2024.
- [4] Y. Zhang, S. Pan, and J. Yu, “TxAllo: Dynamic transaction allocation in sharded blockchain systems,” in *2023 IEEE 39th International Conference on Data Engineering*, ser. ICDE ’23. IEEE, 2023, pp. 721–733.
- [5] X. Wang, B. Li, L. Jia, and Y. Sun, “Orbit: A dynamic account allocation mechanism in sharding blockchain system,” in *2024 IEEE 44th International Conference on Distributed Computing Systems*, ser. ICDCS ’24. IEEE, 2024, pp. 333–344.
- [6] Y. Zhang, S. Pan, and J. Yu, “Mosaic: Client-driven account allocation framework in sharded blockchains,” in *2025 IEEE 45th International Conference on Distributed Computing Systems*, ser. ICDCS ’25. IEEE, 2025, pp. 604–614.
- [7] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang, J. Zhang, and A. Y. Zomaya, “Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering,” in *IEEE INFOCOM 2023–IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.
- [8] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun, “Concordia: Enabling low-conflict distributed transaction scheduling in sharding blockchain via cooperative perception,” in *Proceedings of the ACM Web Conference 2026*, ser. WWW ’26. ACM, 2026, pp. 5493–5502.
- [9] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen, “Adding concurrency to smart contracts,” in *Proceedings of the 36th ACM Symposium on Principles of Distributed Computing*, ser. PODC ’17. ACM, 2017, pp. 303–312.
- [10] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou, “Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing,” in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’23. ACM, 2023, pp. 232–244.
- [11] M. Bartoletti, L. Galletta, and M. Murgia, “A theory of transaction parallelism in blockchains,” *Logical Methods in Computer Science*, vol. 17, no. 4, pp. 10:1–10:30, 2021.

- [12] M. J. Amiri, D. Agrawal, and A. El Abbadi, "ParBlockchain: Leveraging transaction parallelism in permissioned blockchain systems," in *2019 IEEE 39th International Conference on Distributed Computing Systems*, ser. ICDCS '19. IEEE, 2019, pp. 1337–1347.
- [13] Z. Chen, X. Qi, X. Du, Z. Zhang, and C. Jin, "PEEP: A parallel execution engine for permissioned blockchain systems," in *Database Systems for Advanced Applications*, ser. DASFAA '21. Springer, 2021, pp. 341–357.
- [14] T. Xu, Y. Zhong, Y. Zhang, R. Xiong, J. Zhang, G. Xue, and S. Liu, "Vegeta: Enabling parallel smart contract execution in leaderless blockchains," in *22nd USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '25. USENIX Association, 2025, pp. 795–811. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/xu-tianjing>
- [15] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, "Sui Lutris: A blockchain combining broadcast and consensus," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. ACM, 2024, pp. 2606–2620.
- [16] M. J. Amiri, D. Agrawal, and A. El Abbadi, "SharPer: Sharding permissioned blockchains over network clusters," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. ACM, 2021, pp. 76–88.
- [17] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '19. USENIX Association, 2019, pp. 95–112. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/wang-jiaping>
- [18] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai, "OptChain: Optimal transactions placement for scalable blockchain sharding," in *2019 IEEE 39th International Conference on Distributed Computing Systems*, ser. ICDCS '19. IEEE, 2019, pp. 525–535.
- [19] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière, "Shard scheduler: Object placement and migration in sharded account-based blockchains," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies*, ser. AFT '21. ACM, 2021, pp. 43–56.
- [20] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *IEEE INFOCOM 2022—IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1968–1977.
- [21] C. Curino, Y. Zhang, E. P. C. Jones, and S. Madden, "Schism: A workload-driven approach to database replication and partitioning," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 48–57, 2010.